Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Tester une application

Du test unitaire au test end-to-end

Speaker notes

Premier cours de la série. On y présente FraisDirect, l'application fictive qu'on retrouvera dans les cours middleware et sécurité. Les étudiants connaissent déjà JUnit : on ne réexplique pas la syntaxe, on se concentre sur quoi tester, quand, et avec quel outil.

FraisDirect

Une épicerie en ligne qui livre des produits frais



On va la faire grandir dans les prochains cours. Aujourd'hui : s'assurer qu'elle marche.

Speaker notes

Le fil rouge de tous les cours. Ici, elle est encore petite : une application, une base. Le cours middleware la fera grandir, le cours sécurité lui ajoutera des comptes.

Une règle métier

La livraison coûte **4,90 €**

Elle est **offerte dès 50 €** d'achat

Speaker notes

La règle qui va servir d'exemple pour tout le début du cours. Elle a l'air triviale, c'est voulu : même une règle de deux lignes cache un cas limite, on le verra en TDD.

Un lundi matin

1. Un développeur modifie le calcul des promotions
2. Il teste sa promotion à la main : elle marche
3. Il met en production
4. La livraison redevient payante pour tout le monde
5. On l'apprend une semaine plus tard, par les avis clients

Speaker notes

Demander à la salle : qu'est-ce qui a manqué ? Il a testé ce qu'il a modifié, pas ce qui marchait déjà. C'est la définition d'une régression.

Pourquoi tester ?



Speaker notes

Laisser l'image quelques secondes, puis passer à la réponse.

Pourquoi tester ?

- Vérifier que les nouvelles fonctionnalités fonctionnent
- Vérifier qu'il n'y a pas de **régression** : ce qui marchait marche toujours

Speaker notes

Le deuxième point est le plus important et le plus oublié. Une fonctionnalité, on la teste au moins une fois à la main. Ce qui marchait déjà, personne ne le reteste.

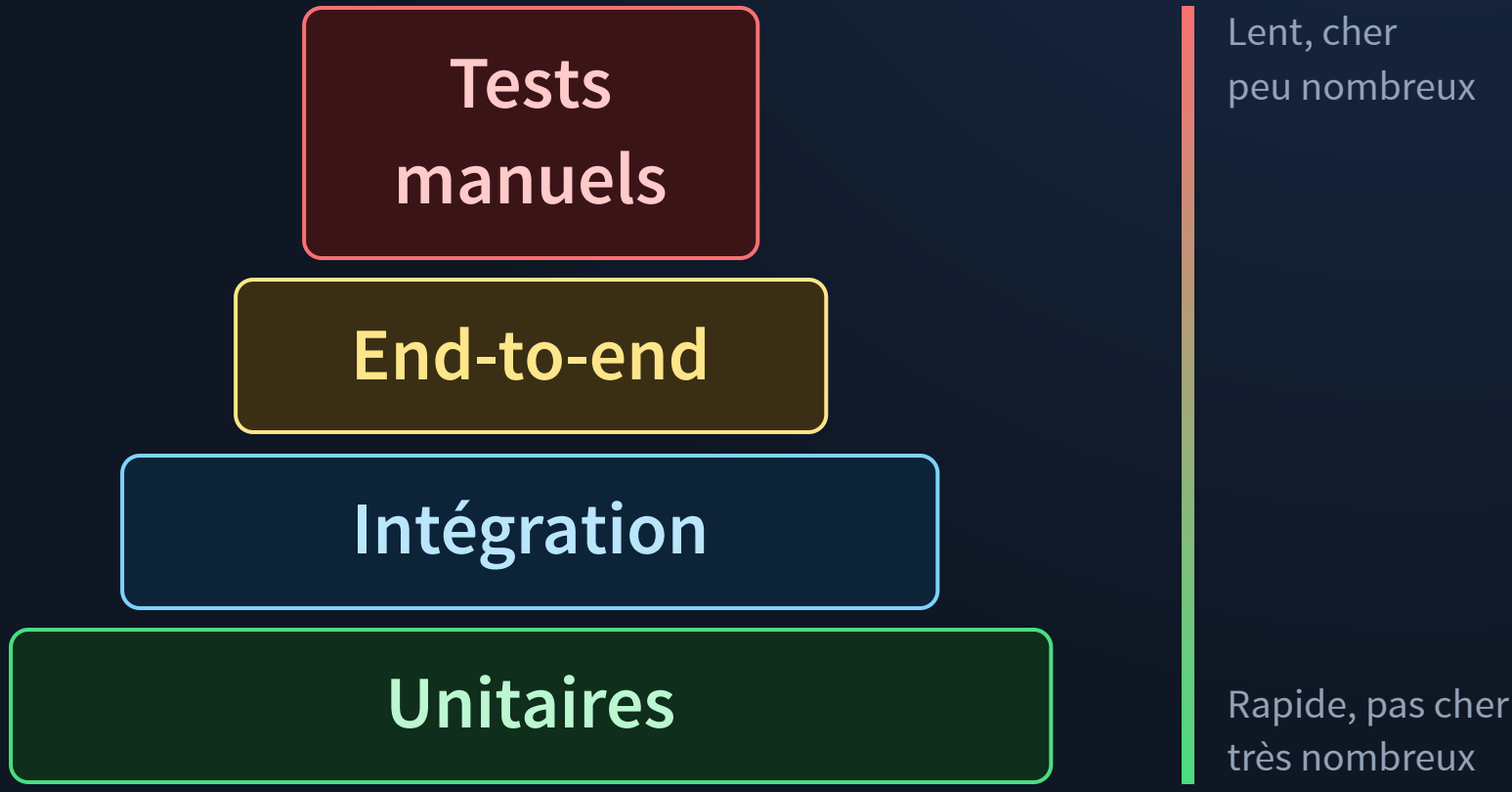
Au programme

1. Quand écrire ses tests : le **TDD**
2. Tester une classe seule : **tests unitaires**, mocks et fakes
3. Tester avec le framework et la base : **tests d'intégration**
4. Tester comme un utilisateur : **tests end-to-end**

Speaker notes

Les trois derniers points sont les trois étages de la pyramide qu'on va voir juste après.

Comment tester ?



Speaker notes

Plus on monte, plus un test vérifie de choses d'un coup, mais plus il est lent à écrire, à exécuter et à réparer. D'où la forme : beaucoup de tests unitaires en bas, quelques tests end-to-end en haut. Les tests manuels ne disparaissent pas, ils deviennent rares.

Pourquoi on gagne du temps

À la main

1. Démarrer l'application
2. Préparer les données
3. Remplir un panier à 49,99 €
4. Lire les frais de livraison
5. Recommencer à 50 €, à 60 €…

Avec des tests

1. Lancer les tests

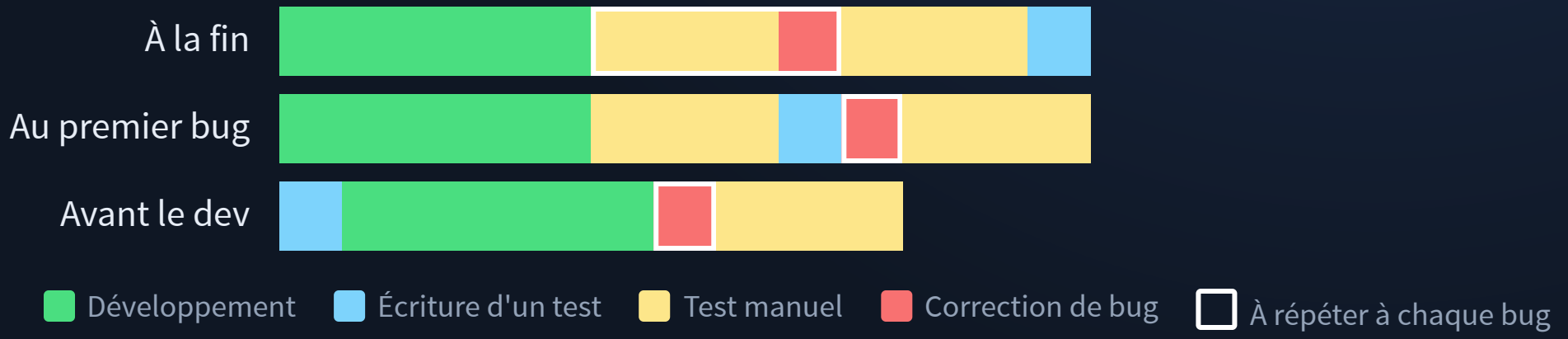
Quelques secondes, tous les cas d'un coup, à chaque modification.

Speaker notes

Demander combien de temps prend la colonne de gauche : quelques minutes. Et on la refait à chaque modification, donc en pratique on ne la refait pas. C'est comme ça qu'arrivent les régressions du lundi matin.

Quand écrire ses tests ?

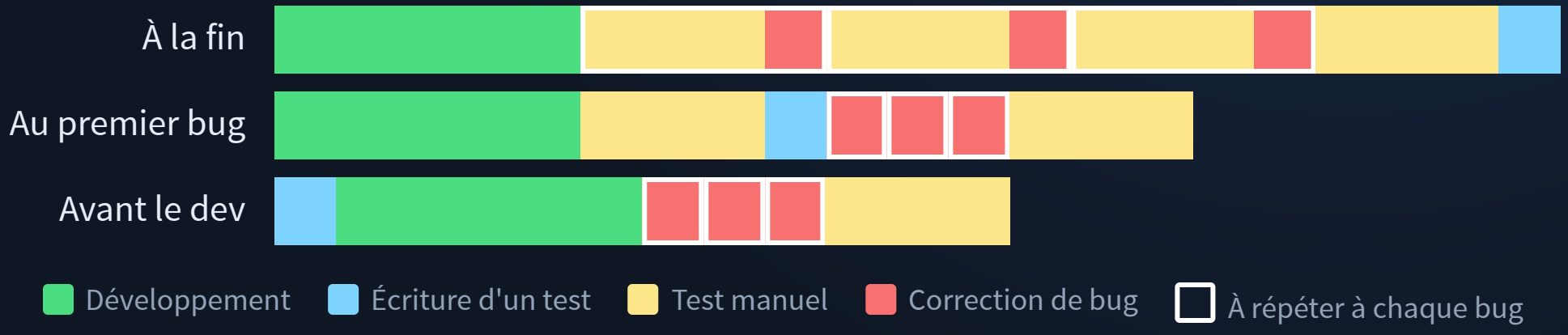
Une fonctionnalité, un bug



Speaker notes

Lire ligne par ligne. Tests à la fin : chaque bug coûte une correction plus un test manuel complet. Au premier bug : on écrit le test à ce moment-là, ensuite le bug ne coûte plus que sa correction. Avant le dev : le test est déjà là, on ne fait le test manuel qu'une fois, à la fin. Pour un seul bug, les trois se valent à peu près.

Et si le bug revient trois fois ?



Speaker notes

Les deux schémas sont à la même échelle. Avec les tests à la fin, chaque bug refait payer un test manuel complet. Avec le test écrit avant, un bug ne coûte que sa correction : le test dit tout de suite si c'est réparé. C'est l'argument du TDD, qui arrive juste après.

ACTE 1

TDD

Test Driven Development : le test d'abord

Speaker notes

On déroule le TDD sur la règle de la livraison, en trois tours. Prendre le temps sur chaque tour : c'est le rythme qu'il faut faire sentir, pas le code, qui est trivial.

QUESTION

En TDD, par quoi commence-t-on ?

A. Écrire le code de la fonctionnalité

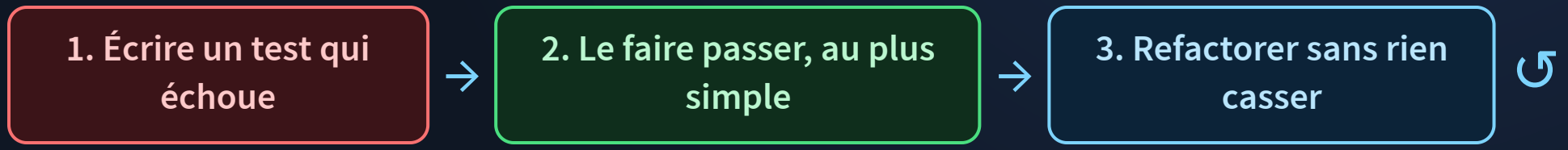
B. Écrire un test qui échoue

C. Refactorer le code existant

Speaker notes

Réponse B. Poser la question avant d'expliquer : beaucoup répondent A, et c'est justement ce que le TDD inverse.

Le cycle



- **Rouge** : le test échoue, il décrit ce qu'on veut
- **Vert** : le minimum de code pour qu'il passe
- **Refactor** : on nettoie, les tests protègent

Speaker notes

Insister sur « le minimum ». Les étudiants voudront écrire toute la règle d'un coup. Le TDD, c'est des petits pas : chaque test ajoute une seule exigence.

Tour 1 · Rouge

Un panier de 30 € paie la livraison

```java
class FraisLivraisonTest {

    @Test
    void unPanierDe30EurosPaieLaLivraison() {
        // Given : les montants sont en centimes
        long totalPanier = 3000;
        // When
        long frais = new FraisLivraison().calculer(totalPanier);
        // Then
        assertEquals(490, frais);
    }
}
```

Speaker notes

Rappel JUnit au passage : Given / When / Then, un nom de test qui se lit comme une phrase. Les montants sont en centimes pour éviter les erreurs d'arrondi des double. La classe FraisLivraison n'existe pas encore : le test ne compile même pas. C'est rouge.

Tour 1 · Rouge

On crée la classe, juste pour que ça compile

```
class FraisLivraison {
    long calculer(long totalPanier) {
        return 0;
    }
}
```

```
✗ unPanierDe30EurosPaieLaLivraison
expected: <490> but was: <0>
```

Speaker notes

On lance le test et on le voit échouer, pour la bonne raison. Un test qu'on n'a jamais vu rouge peut très bien ne rien tester.

Tour 1 · Vert

Le minimum pour passer au vert

```
1 class FraisLivraison {
2     long calculer(long totalPanier) {
3         return 490;
4     }
5 }
```

✓ unPanierDe30EurosPaieLaLivraison

Speaker notes

Les étudiants vont trouver ça ridicule, et c'est le but de la discussion : rien dans les tests ne demande autre chose pour l'instant. C'est le test suivant qui va forcer la vraie règle.

Tour 2 · Rouge

Un panier de 60 € a la livraison offerte

```
@Test
void unPanierDe60EurosALaLivraisonOfferte() {
    long frais = new FraisLivraison().calculer(6000);

    assertEquals(0, frais);
}
```

```
✗ unPanierDe60EurosALaLivraisonOfferte
expected: <0> but was: <490>
```

Speaker notes

Le nouveau test échoue, le premier passe toujours.

Tour 2 · Vert

```
class FraisLivraison {
    long calculer(long totalPanier) {
        if (totalPanier > 5000) {
            return 0;
        }
        return 490;
    }
}
```

✓ unPanierDe30EurosPaieLaLivraison
✓ unPanierDe60EurosALaLivraisonOfferte

Speaker notes

Ne pas relever le > tout de suite : la question qui suit le fait.

QUESTION

Avec ce code, un panier de pile 50 € a-t-il la livraison offerte ?

```
if (totalPanier > 5000) return 0;
```

Speaker notes

Réponse : non. La règle dit « dès 50 € », donc 50 € compris. Les deux tests sont verts et pourtant le code est faux : aucun test ne regarde la limite. Les valeurs limites sont là où se cachent les bugs.

Tour 3 · Rouge

On teste la limite

```
@Test
void unPanierDePile50EurosALaLivraisonOfferte() {
    long frais = new FraisLivraison().calculer(5000);

    assertEquals(0, frais);
}
```

```
✗ unPanierDePile50EurosALaLivraisonOfferte
expected: <0> but was: <490>
```

Speaker notes

Le bug est trouvé par un test, avant la production, et non par les avis clients.

Tour 3 · Vert

```
1 class FraisLivraison {
2     long calculer(long totalPanier) {
3         if (totalPanier >= 5000) {
4             return 0;
5         }
6         return 490;
7     }
8 }
```

✓ unPanierDe30EurosPaieLaLivraison
✓ unPanierDe60EurosALaLivraisonOfferte
✓ unPanierDePile50EurosALaLivraisonOfferte

Speaker notes

Un seul caractère change. Les trois tests sont verts.

Refactor

On donne un nom aux nombres magiques

```java
class FraisLivraison {
    static final long SEUIL_LIVRAISON_OFFERTE = 5000;
    static final long FRAIS_LIVRAISON = 490;

    long calculer(long totalPanier) {
        if (totalPanier >= SEUIL_LIVRAISON_OFFERTE) {
            return 0;
        }
        return FRAIS_LIVRAISON;
    }
}
```

✓ 3 tests passent : le refactor n'a rien cassé

Speaker notes

C'est l'étape qu'on saute toujours, et c'est elle qui rend le code lisible. On peut la faire sans peur parce que les tests vérifient qu'on n'a rien cassé.

Ce qu'on a gagné

- La règle est écrite trois fois, dont deux en langage clair : les noms des tests
- Le cas limite a été trouvé avant la production
- Le prochain qui touche à ce code saura tout de suite s'il casse la règle

Speaker notes

Revenir à l'histoire du lundi matin : avec ces trois tests, le développeur des promotions aurait vu un test rouge avant de mettre en production.

Mon (vrai) avis sur le TDD

- ⊕ Essentiel pour le débogage
- ⊕ Pratique pour des POC techniques
- ⊖ Compliqué pour réfléchir à une architecture

Speaker notes

Pour corriger un bug, écrire d'abord le test qui le reproduit est presque toujours le bon réflexe. Pour concevoir une architecture qu'on ne connaît pas encore, écrire les tests d'abord est plus difficile : on ne sait pas encore quelles classes existeront.

ACTE 2

Les tests unitaires

Tester une classe, et seulement elle

Speaker notes

FraisLivraison n'avait aucune dépendance, c'était facile. Une vraie classe métier appelle une base, d'autres services, des API externes. Comment la tester seule ?

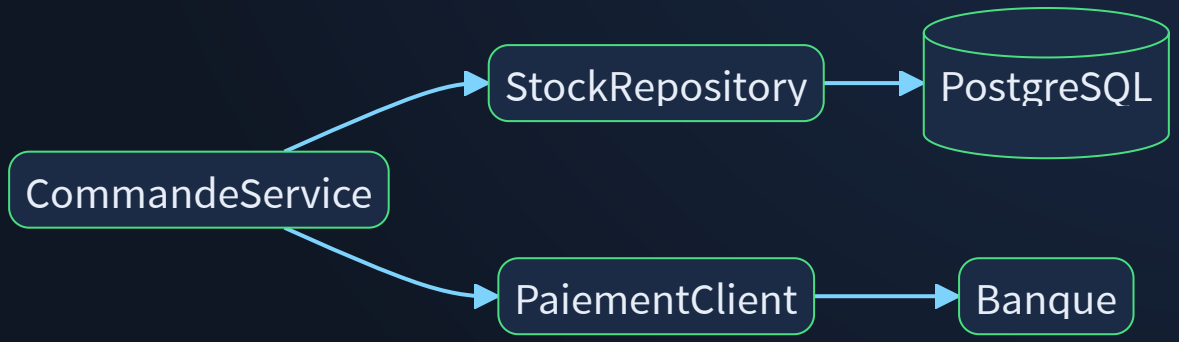
Passer une commande

```
1  class CommandeService {
2      void passerCommande(Commande commande) {
3          String produit = commande.produit();
4          int disponible = stockRepository.quantiteDisponible(produit);
5          if (disponible < commande.quantite()) {
6              throw new StockInsuffisantException(produit);
7          }
8          paiementClient.payer(commande.client(),
9                  commande.montant());
10     }
11 }
```

Speaker notes

La règle à tester : si le stock ne suffit pas, on refuse la commande et on ne débite pas le client. Lire le code ligne par ligne avec les fragments. StockRepository et PaiementClient sont injectés par le constructeur.

Le problème

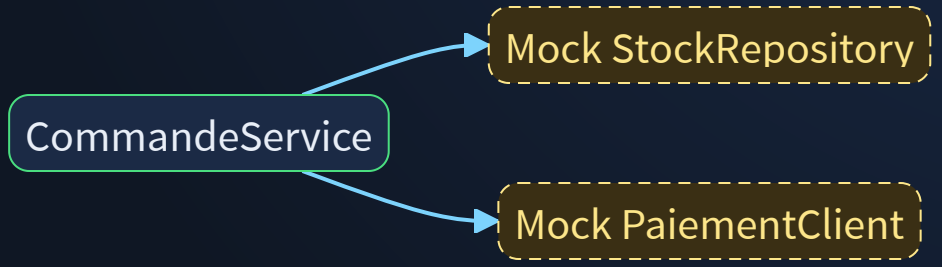


- Il faut une base remplie avec le bon stock
- Il faut une banque… qui débite vraiment
- Et on veut juste tester un `if`

Speaker notes

Demander à la salle comment ils feraient. On veut tester la règle de CommandeService, pas PostgreSQL ni la banque.

La solution : des faux



- Un **mock** est un faux objet
- On lui dit quoi répondre
- On vérifie ensuite comment il a été appelé

Speaker notes

Le service ne voit pas la différence : il reçoit un objet qui implémente la même interface. C'est pour ça qu'on injecte les dépendances par le constructeur.

C'est quoi un mock ?



Speaker notes

Pause détente avant les quatre étapes avec Mockito.

Étape 1 · Créer les faux

```java
@ExtendWith(MockitoExtension.class)
class CommandeServiceTest {

    @Mock
    private StockRepository stockRepository;
    @Mock
    private PaiementClient paiementClient;

    @InjectMocks
    private CommandeService commandeService;
}
```

- `@Mock` crée un faux qui ne fait rien
- `@InjectMocks` crée le vrai service et lui donne les faux

Speaker notes

Piège classique : sans @ExtendWith(MockitoExtension.class), les annotations sont ignorées, les champs restent null et le test plante avec une NullPointerException. Par défaut, un mock renvoie 0, null ou une liste vide.

Étape 2 · Programmer le faux

```
// Given : il ne reste que 2 tomates
when(stockRepository.quantiteDisponible("tomate"))
        .thenReturn(2);
Commande commande = new Commande("alice", "tomate", 3, 1200);
```

« Quand on te demande le stock de tomates, réponds 2. »

Speaker notes

Lire le when / thenReturn à voix haute comme une phrase. Alice commande 3 tomates pour 12 €, il n'en reste que 2.

Étape 3 · Vérifier le résultat

```
// When / Then : la commande est refusée
assertThrows(StockInsuffisantException.class,
        () -> commandeService.passerCommande(commande));
```

Le service a bien appelé le faux, reçu 2, et refusé.

Speaker notes

assertThrows exécute la lambda et vérifie qu'elle lève bien cette exception. Si elle n'en lève pas, le test échoue.

Étape 4 · Vérifier les appels

```
// Then : Alice n'a pas été débitée
verify(paiementClient, never()).payer(any(), anyLong());
```

Le résultat ne suffit pas : on vérifie aussi ce qui **ne** s'est **pas** passé.

Speaker notes

Si quelqu'un déplace l'appel à payer avant le contrôle du stock, l'exception est toujours levée, mais Alice est débitée. Seul ce verify le détecte.

Le test complet

```java
@Test
void refuseLaCommandeSiLeStockManque() {
    when(stockRepository.quantiteDisponible("tomate"))
            .thenReturn(2);
    Commande commande = new Commande("alice", "tomate", 3, 1200);

    assertThrows(StockInsuffisantException.class,
            () -> commandeService.passerCommande(commande));

    verify(paiementClient, never()).payer(any(), anyLong());
}
```

Speaker notes

Les trois blocs Given / When / Then restent visibles grâce aux lignes vides.

Le piège : tester le mock

```
when(additionService.add(2, 3)).thenReturn(5);

assertEquals(5, calculator.calculate(2, 3));
```

- ➖ `calculate` ne fait que renvoyer ce que dit le service
- ➖ On vérifie que le mock renvoie… ce qu'on lui a dit de renvoyer
- ➕ Un test unitaire vérifie une **décision** de la classe : un `if`, un calcul, un appel ou un non-appel

Speaker notes

Ce test est toujours vert, quoi que fasse calculate tant qu'elle passe la main. Demander : quelle décision CommandeService prend-elle ? Refuser ou payer. C'est ça qu'on teste.

QUESTION

Pour vérifier qu'Alice n'a pas été débitée, de quoi a-t-on besoin ?

A. D'un compte de test à la banque

B. D'un mock de `PaiementClient` et de `verify`

C. D'un `assertEquals` sur le retour de `passerCommande`

Speaker notes

Réponse B. C ne marche pas : passerCommande ne renvoie rien, l'effet est un appel à un autre objet. A : jamais de vrai service externe dans un test.

C'est quoi un fake ?

- Une vraie implémentation, mais simplifiée
- Exemple : un repository qui stocke dans une `Map`

```
class StockRepositoryEnMemoire implements StockRepository {
    private final Map<String, Integer> stock = new HashMap<>();

    void ajouter(String produit, int quantite) {
        stock.put(produit, quantite);
    }

    @Override
    public int quantiteDisponible(String produit) {
        return stock.getOrDefault(produit, 0);
    }
}
```

Speaker notes

Pas de Mockito : c'est une classe normale, écrite une fois et réutilisée dans tous les tests. Dans le test : stock.ajouter("tomate", 2), puis new CommandeService(stock, paiementClient).

Mock ou fake ?

	Mock	Fake
Comment	Mockito, dans chaque test	Une classe écrite une fois
Répond	Ce qu'on lui a programmé	Comme le vrai, en simplifié
Vérifie les appels	Oui, avec `verify`	Non
Idéal pour	Services externes, appels à vérifier	Repositories, données manipulées

Speaker notes

Les deux se combinent très bien : un fake pour le stock, un mock pour le paiement qu'on veut vérifier.

QUESTION

À vous : le stock suffit, Alice est débitée de 12 €

Programmez le stock, passez la commande, vérifiez le paiement.

Speaker notes

Réponse attendue : when(stockRepository.quantiteDisponible("tomate")).thenReturn(5); commandeService.passerCommande(new Commande("alice", "tomate", 3, 1200)); verify(paiementClient).payer("alice", 1200); Erreurs fréquentes à regrouper : oublier le when (le mock renvoie 0, donc exception), vérifier avec assertEquals au lieu de verify, ou verify sans les bons arguments.



Speaker notes

Pause après l'exercice.

EXERCICE

Feature

- Faire un virement d'un montant X entre deux comptes
- Retirer X au compte source et ajouter X au compte cible
- Le montant du virement doit être positif
- Si le compte source finit avec moins de 0, on envoie un montant de 0,5 vers le compte de la banque

Speaker notes

À faire en TDD : un test rouge par règle, dans l'ordre. Le dernier point est le cas intéressant pour les mocks : le compte de la banque.

EXERCICE

Play time

Speaker notes

Les étudiants codent la feature en TDD.

EXERCICE

Play time Angular

Speaker notes

Même démarche côté front.

ACTE 3

Les tests d'intégration

Tester avec le framework et la base

Speaker notes

Les tests unitaires ont remplacé la base par des faux. Mais qui vérifie que les requêtes SQL marchent, que le JSON est bien lu, que Spring câble bien tout ? C'est le rôle des tests d'intégration.

Le principe

- Lancer tout ou une partie de l'application
- Avec une vraie base de données
- Quelques faux, seulement pour les services externes

Speaker notes

La différence avec l'unitaire : on garde les vraies dépendances internes, on ne remplace que ce qui sort de chez nous.

Pourquoi ?

- Tester la communication entre les classes
- Tester la communication avec le framework
- Vérifier les requêtes sur la vraie base

Speaker notes

Exemple : une faute de frappe dans le nom d'une méthode de repository Spring Data. Les tests unitaires sont verts, ils utilisent un mock. L'application ne démarre pas.

Alors, que des tests d'intégration ?

- C'est long à exécuter
- C'est plus compliqué à maintenir
- C'est plus compliqué à déboguer

Speaker notes

Un test unitaire tourne en millisecondes, un test d'intégration en secondes. Et quand il échoue, la cause peut être dans n'importe quelle couche. Retour à la pyramide : peu de tests d'intégration, beaucoup d'unitaires.

Trois niveaux

Test de…	Ce qu'on vérifie	Outil
Repository	Les requêtes sur la base	`@DataJpaTest`
Service	Le framework, la base, les transactions	`@SpringBootTest`
Contrôleur	L'URL, le JSON, la sécurité, les codes HTTP	`@WebMvcTest`

Speaker notes

On va dérouler le repository et le contrôleur. Le test de service démarre toute l'application, il s'écrit comme le test de repository avec @SpringBootTest.

H2 : la base qui n'est pas PostgreSQL

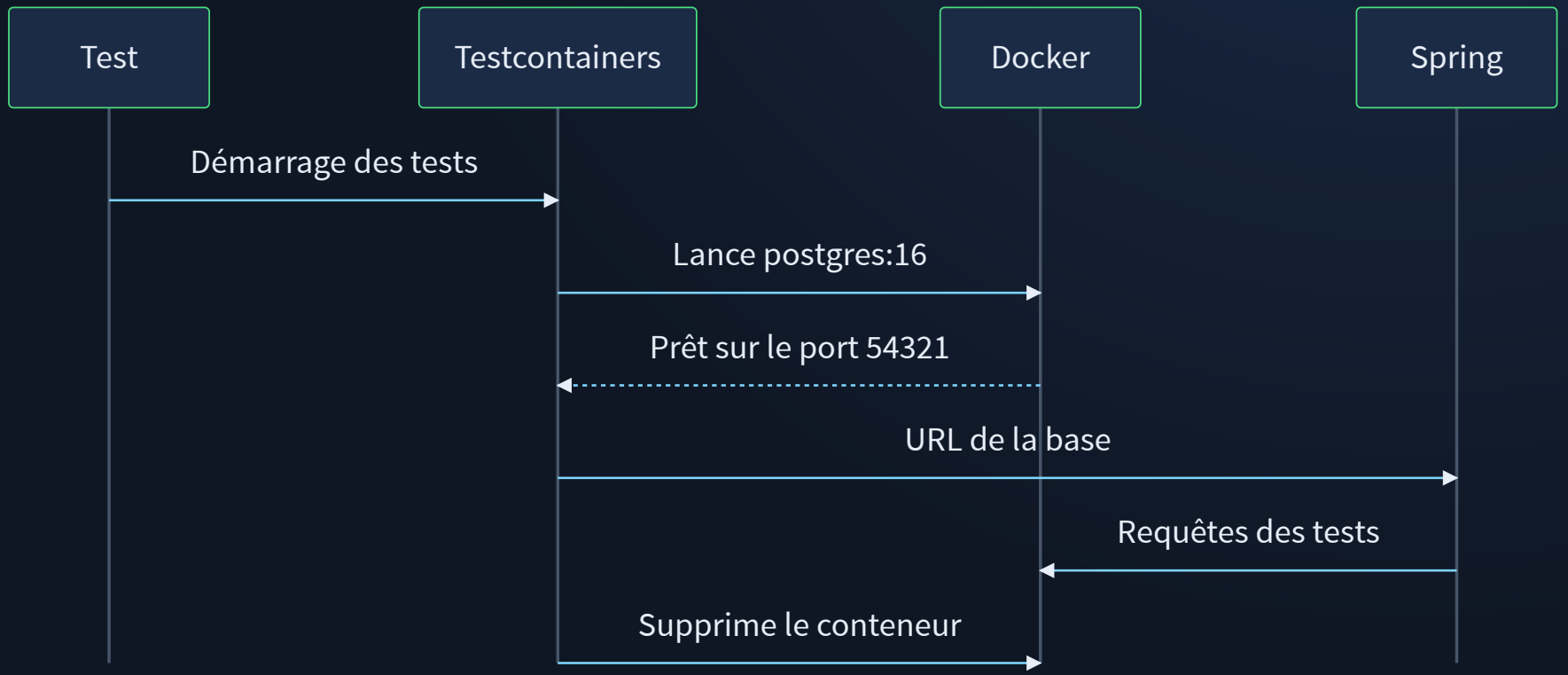
- Longtemps, on a utilisé **H2** : une base SQL en mémoire
- ⊕ Rapide, rien à installer
- ⊖ Ce n'est pas PostgreSQL : types, fonctions et SQL diffèrent
- ⊖ Test vert sur H2, erreur en production

Speaker notes

Exemple : une requête native avec ILIKE ou un champ jsonb marche en production sur PostgreSQL et plante sur H2, ou l'inverse. Le test ne prouve rien sur la vraie base.

Testcontainers

Un vrai PostgreSQL, dans Docker, le temps des tests



Speaker notes

Dérouler les flèches une par une. Le port est choisi au hasard pour ne pas entrer en conflit avec un PostgreSQL local. Il faut Docker sur le poste et dans la CI.

Test de repository

```java
@DataJpaTest
@Testcontainers
@AutoConfigureTestDatabase(replace = Replace.NONE)
class ProduitRepositoryTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
            new PostgreSQLContainer<>("postgres:16");

    @Autowired
    private ProduitRepository produitRepository;

    @Test
    void trouveLesProduitsEnRupture() {
        produitRepository.save(new Produit("tomate", 0));
        produitRepository.save(new Produit("comté", 4));

        List<Produit> enRupture = produitRepository.findByStock(0);

        assertEquals(1, enRupture.size());
        assertEquals("tomate", enRupture.get(0).getNom());
```

Speaker notes

Étapes : @DataJpaTest ne démarre que JPA ; replace = NONE l'empêche de remplacer la base par une base en mémoire. @ServiceConnection (Spring Boot 3.1 et plus) donne l'URL du conteneur à Spring, sans configuration. Ensuite un test classique Given / When / Then, sur une vraie base.

Et pour un middleware ?

RabbitMQ, Kafka, Redis… : pareil, avec Testcontainers

```
@Container
@ServiceConnection
static RabbitMQContainer rabbit =
        new RabbitMQContainer("rabbitmq:4-management");
```

Speaker notes

On reverra RabbitMQ dans le cours middleware. Le principe reste le même : un vrai serveur dans Docker, le temps des tests.

Test de contrôleur

1. `@WebMvcTest` ne démarre que la couche web
2. Le service est un mock : on teste le mapping, pas la règle métier
3. `MockMvc` simule une requête HTTP, sans vrai serveur
4. On vérifie le code HTTP et le JSON

Speaker notes

Ce qu'on vérifie ici : l'URL, la lecture des paramètres, le JSON produit, les codes d'erreur. La logique, elle, est déjà couverte par les tests unitaires du service.

Test de contrôleur

```java
@WebMvcTest(CommandeController.class)
class CommandeControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private CommandeService commandeService;

    @Test
    void renvoieLaCommandeEnJson() throws Exception {
        when(commandeService.trouver(42))
                .thenReturn(new CommandeDto(42, "alice", 1200));

        mockMvc.perform(get("/api/commandes/42"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.client").value("alice"))
                .andExpect(jsonPath("$.montant").value(1200));
    }
}
```

Speaker notes

@MockitoBean existe depuis Spring Boot 3.4 ; avant, c'était @MockBean. Même rôle que @Mock, mais le faux est placé dans le contexte Spring. Bon exercice oral : comment tester le cas 404, quand la commande n'existe pas ?

QUESTION

Un test qui démarre l'application avec un PostgreSQL Testcontainers et appelle `GET /api/produits` en JSON, c'est un test…

A. unitaire

B. d'intégration

C. end-to-end

Speaker notes

Réponse B. Il y a une vraie base et le framework, mais pas de navigateur ni de front : ce n'est pas un end-to-end.

Un client HTTP vers un fournisseur

FraisDirect lit le prix du jour sur l'API de la ferme

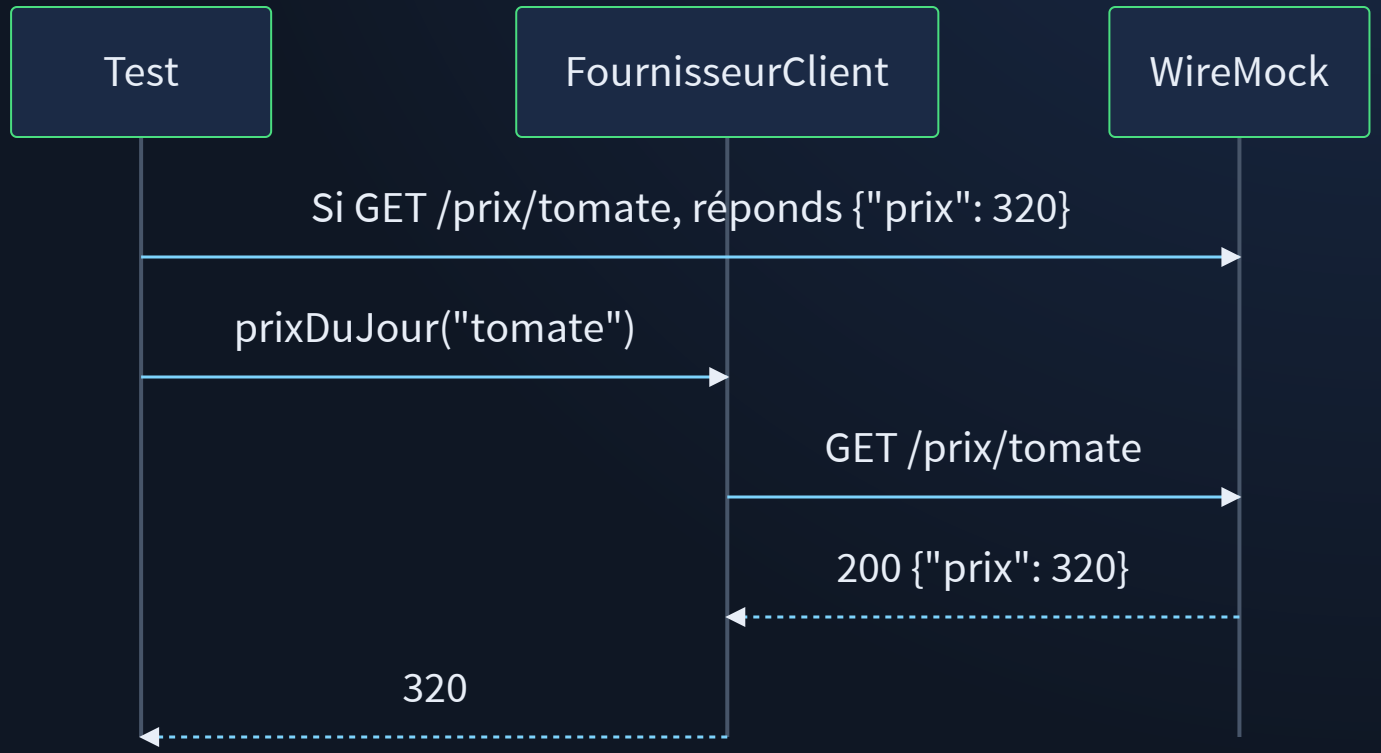
Jamais de vraie requête vers un serveur externe dans un test

- Le serveur peut être lent ou en panne : le test échoue sans raison
- L'API peut être payante, ou créer de vraies commandes

Speaker notes

Un test doit donner le même résultat à chaque exécution. Un serveur qu'on ne contrôle pas rend le test aléatoire.

WireMock : un faux serveur



Speaker notes

Le client ne sait pas qu'il parle à un faux : on lui donne juste une autre URL. Le vrai code HTTP est exécuté : la requête, la lecture du JSON, la gestion des erreurs.

WireMock : le test

```
1  @WireMockTest
2  class FournisseurClientTest {
3
4      @Test
5      void litLePrixDuJour(WireMockRuntimeInfo wiremock) {
6          stubFor(get("/prix/tomate")
7                  .willReturn(okJson("{\"prix\": 320}")));
8          var url = wiremock.getHttpBaseUrl();
9          var client = new FournisseurClient(url);
10
11         long prix = client.prixDuJour("tomate");
12
13         assertEquals(320, prix);
14     }
15 }
```

Speaker notes

Étapes : programmer la réponse, pointer le client vers WireMock, appeler, vérifier. Le vrai intérêt : tester les cas qu'on ne peut pas provoquer chez le fournisseur, comme une erreur 500 ou un délai de 30 secondes.

ACTE 4

Les tests end-to-end

Tester comme un utilisateur

Speaker notes

Le haut de la pyramide : tout est vrai, du navigateur à la base.

Le principe

1. Lancer l'application complète : front, back, middlewares
2. Simuler des actions dans un navigateur
3. Vérifier ce que l'utilisateur voit

Objectif : vérifier que **tout communique** correctement

Speaker notes

Un robot clique à la place de l'utilisateur. Le test ne connaît pas le code, seulement l'écran.

Un scénario

1. Ouvrir FraisDirect
2. Ajouter deux comtés à 27 € au panier
3. Ouvrir le panier
4. Lire « Livraison offerte »

Speaker notes

La même règle que le TDD, vue depuis l'écran. Ce test vérifie que le front affiche bien ce que l'API calcule.

Exemple avec Playwright

```
1  test('livraison offerte dès 50 €', async ({ page }) => {
2    await page.goto('http://localhost:4200');
3    const comte = page.getByRole('button',
4        { name: 'Ajouter Comté 18 mois' });
5    await comte.click();
6    await comte.click();
7    await page.getByRole('link', { name: 'Panier' }).click();
8    await expect(page.getByText('Livraison offerte'))
9        .toBeVisible();
10 });
```

Selenium et Cypress fonctionnent sur le même principe

Speaker notes

Chaque ligne correspond à une étape du scénario. On cherche les éléments comme un utilisateur, par leur rôle et leur texte, pas par leur classe CSS : le test casse moins souvent quand le design change.

Les difficultés

- Préparer la base
- Lourd à écrire
- Lourd à exécuter
- Quand les exécuter ?
- Difficile à maintenir

Speaker notes

Préparer la base : le comté doit exister, coûter 27 €, être en stock. Quand les exécuter : trop lents pour chaque commit, souvent la nuit ou avant une mise en production.

Faut-il en faire ?

- ⊕ **Oui !**
- ⊕ Au moins sur les chemins usuels : commander, payer, se connecter
- ⊖ Ils ne remplacent pas les tests manuels

Speaker notes

Peu de tests, mais sur ce qui rapporte : si la commande ne marche plus, FraisDirect ne gagne plus d'argent.

À retenir

- Tester, c'est surtout éviter les **régressions**
- TDD : rouge, vert, refactor, par petits pas
- Unitaire : une classe, des mocks et des fakes, tester ses décisions
- Intégration : une vraie base avec Testcontainers, jamais de vrai serveur externe
- End-to-end : peu, sur les chemins qui comptent

Speaker notes

Revenir à la pyramide une dernière fois. La prochaine fois, FraisDirect grandit : cours middleware.